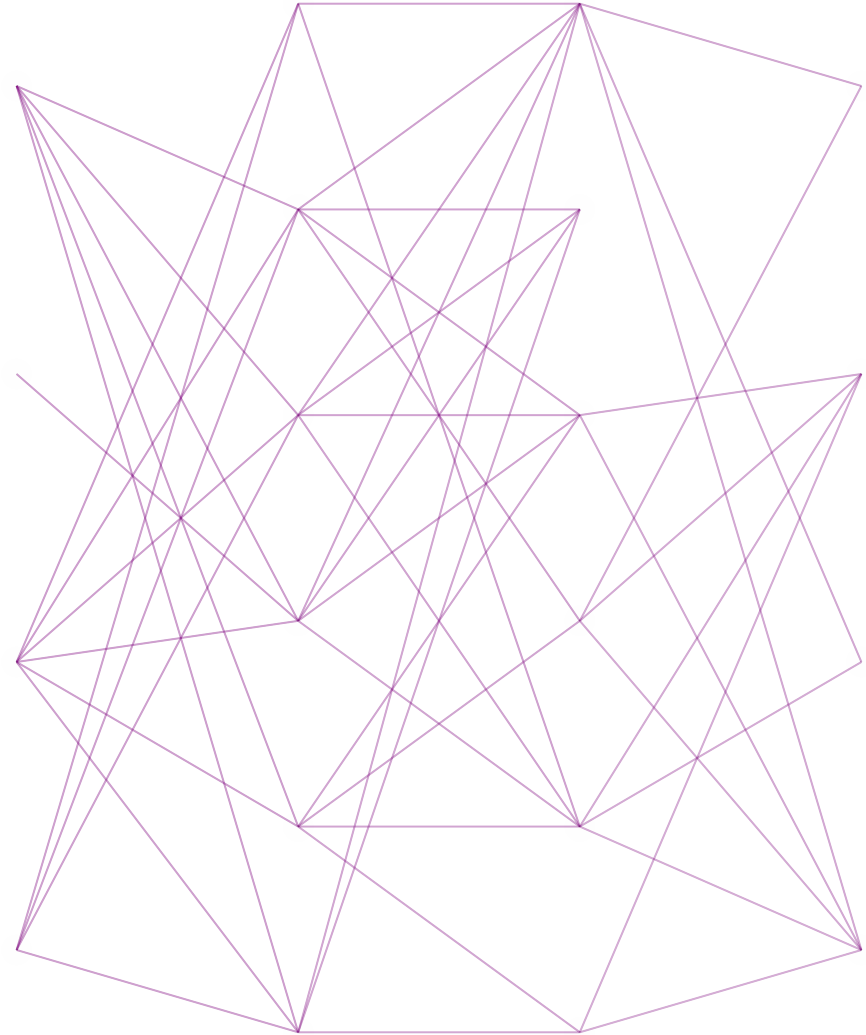


Neural IR & Retrieval- Augmented Generation

From BM25 to Dense Vectors
Transformers, BERT, and RAG



Limitations of Symbolic IR (BM25)

✗ Vocabulary Mismatch

Query: "automobile repair"
Doc: "car mechanic shop"

Score = 0.0

Symbolic models (TF-IDF) require exact string overlap. They don't know that "car" $\approx$ "automobile".



⚠ Polysemy (Ambiguity)

Query: "jaguar speed"
Doc 1: "The cat runs fast"
Doc 2: "The car drives fast"

Score = High (for both)

Keywords are identical, but meanings differ. The model ignores context.



*"We are matching **strings**, not **things**."*

The Semantic Gap



User Intent

"I want affordable Japanese food."
(Concepts, Needs, Feelings)

THE GAP



System Index

```
{  
  "cheap": [D1, D4],  
  "sushi": [D2, D9]  
}
```

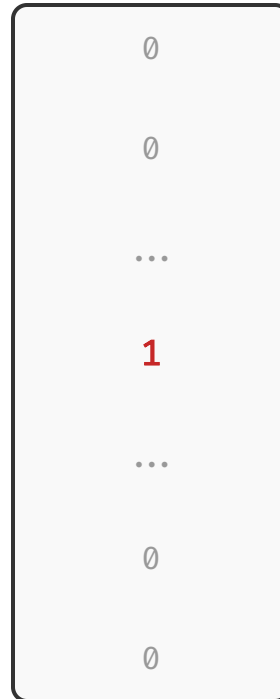
The Neural Solution:

Map both the *Intent* and the *Document* to the same high-dimensional vector space, where "affordable" is close to "cheap".

Word Embeddings: The Foundation

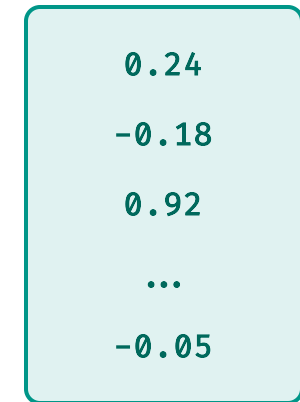
One-Hot Encoding

"Hotel" =



Dense Embedding

"Hotel" $\approx$



Dimension: ~100,000

Type: Sparse

Meaning

Dimension: ~300 - 768

Type: Dense (Float)

Meaning: Semantic

"You shall know a word by the company it keeps." — J.R. Firth (1957)

Word2Vec & GloVe: Static Embeddings

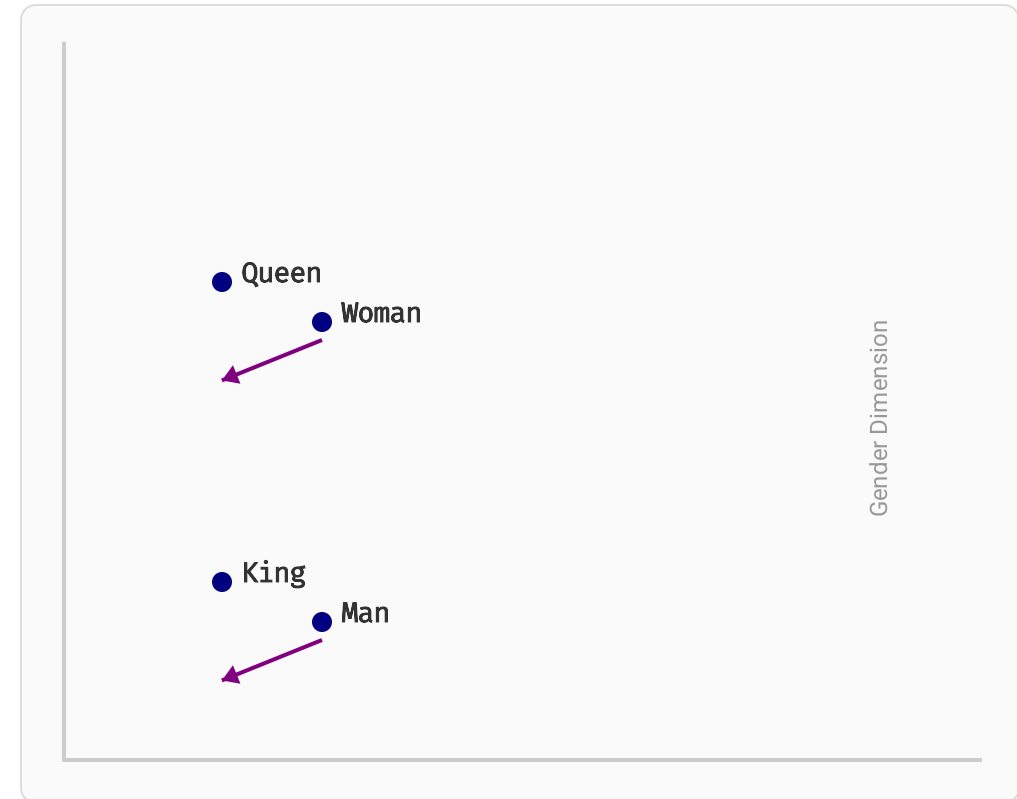
"You shall know a word by the company it keeps."
— J.R. Firth (1957)

Word2Vec (2013)

A neural network that learns vector representations by predicting surrounding words (Skip-gram) or the center word (CBOW).

GloVe (2014)

Global Vectors. Uses matrix factorization of the entire corpus co-occurrence matrix.



$$\text{vec}(\text{King}) - \text{vec}(\text{Man}) + \text{vec}(\text{Woman}) \approx \text{vec}(\text{Queen})$$

! **Limitation: Static Embeddings are Context-Free.** "Bank" (river) and "Bank" (money) get the exact same vector.

Contextual Embeddings (BERT)

Static (Word2Vec / GloVe)

One Word = One Vector



Bank

The vector is a confusing average of all meanings.

Contextual (BERT)

Vector Depends on Sentence

"I sat by the **bank**."

→ $\text{vec}_1 \approx [0.1, 0.9, \dots]$ (Water)

"I went to the **bank**."

→ $\text{vec}_2 \approx [0.8, 0.2, \dots]$ (Finance)

$$E(\text{word}) = f(\text{word}, \text{context})$$

Transformer Architecture for IR

Self-Attention Mechanism

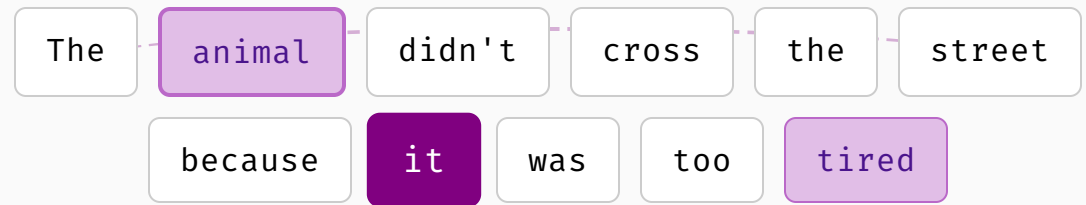
Unlike RNNs which read sequentially (left-to-right), Transformers read the entire sentence at once.

Every word attends to **every other word** to understand its role in the specific context.

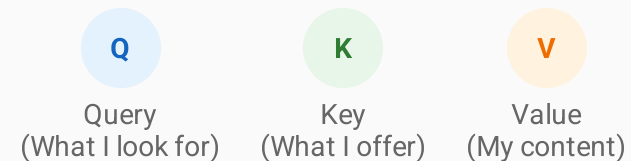
Context Disambiguation

To understand "it", the model looks strongly at "animal" and weakly at "street".

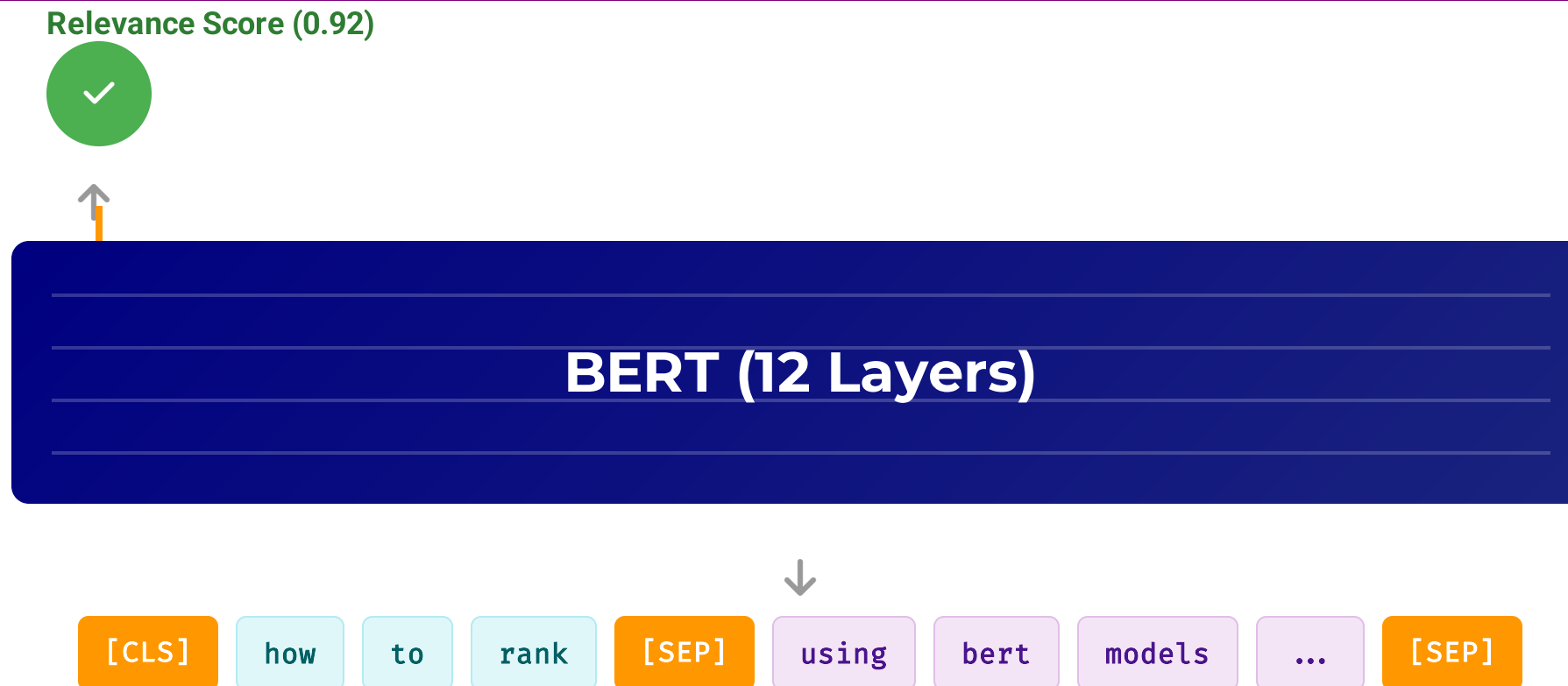
"The animal didn't cross the street because it was too tired."



$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k})V$$



BERT for Ranking



1. Input Construction

Concatenate Query and Document with special separators. The [CLS] token is prepended to capture the aggregate representation.

2. Deep Interaction

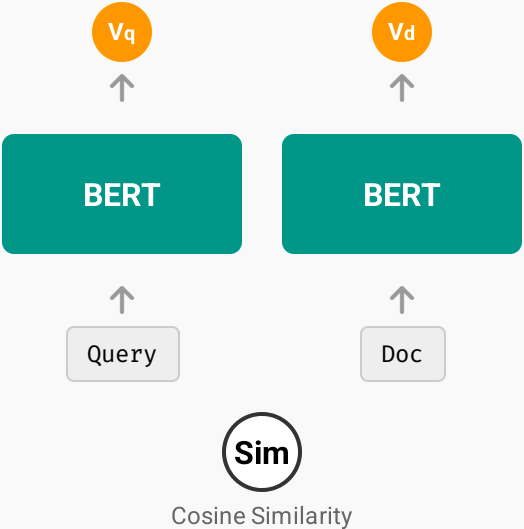
Inside BERT, every query word attends to every document word (Self-Attention). This captures complex relationships like negation or synonymy.

3. Scoring

The final vector of the [CLS] token is fed into a simple classification layer (Linear + Sigmoid) to output a probability.

Bi-Encoders vs. Cross-Encoders

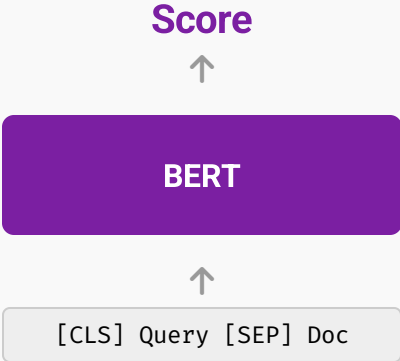
Bi-Encoder



The diagram illustrates the Bi-Encoder architecture. It features two parallel BERT encoders. The left encoder takes a 'Query' input and produces a vector V_q . The right encoder takes a 'Doc' input and produces a vector V_d . These two vectors are then compared using a 'Sim' (Cosine Similarity) operation, represented by a circle with the word 'Sim' inside.

Speed	Very Fast (ANN)
Accuracy	Lower
Interaction	Shallow (Dot Product)

Cross-Encoder

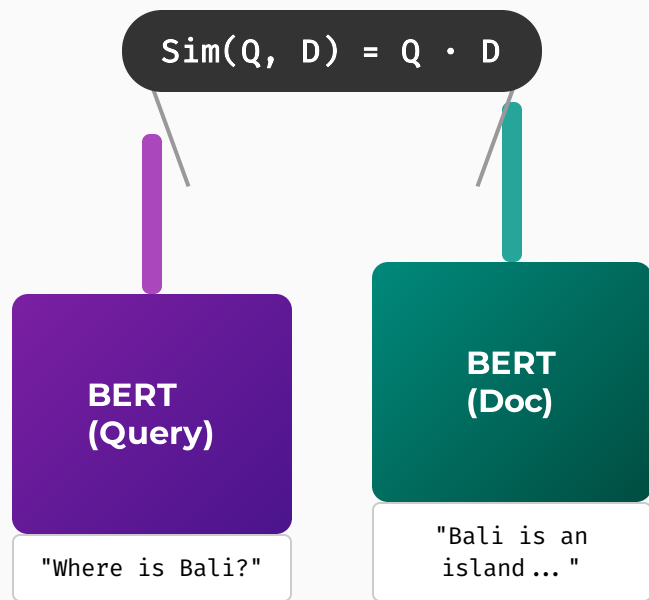


The diagram illustrates the Cross-Encoder architecture. It features a single BERT encoder that takes a concatenated input consisting of a [CLS] token, the 'Query', a [SEP] token, and the 'Doc'. The encoder produces a 'Score' output.

Speed	Slow (No Indexing)
Accuracy	State-of-the-Art
Interaction	Deep (Self-Attention)

Standard Pipeline: Retrieve with Bi-Encoder (Top-100) → Re-rank with Cross-Encoder (Top-10)

Dense Retrieval (DPR)



"Two-Tower" Architecture

Shared Vector Space

Both the Query and the Document are mapped to the **same** high-dimensional space (e.g., 768 dimensions). Relevance is simply geometric proximity.

Offline Indexing

We pre-compute vectors for all 10M+ documents and store them in a Vector Database (FAISS). This is the heavy lifting.

Online Efficiency

At runtime, we only encode the **Query** (once). Retrieval is then a fast Nearest Neighbor search.

 **Result: Semantic Search at Scale (ms latency).**

Approximate Nearest Neighbor (ANN)

The Scaling Problem

Exact Search (k-NN) requires comparing the query vector against *every* document vector ($O(N)$).

For 100M vectors, this takes seconds. We need milliseconds.

The ANN Solution

Partition the vector space into cells (Voronoi) or graphs.
Only search the most promising regions.

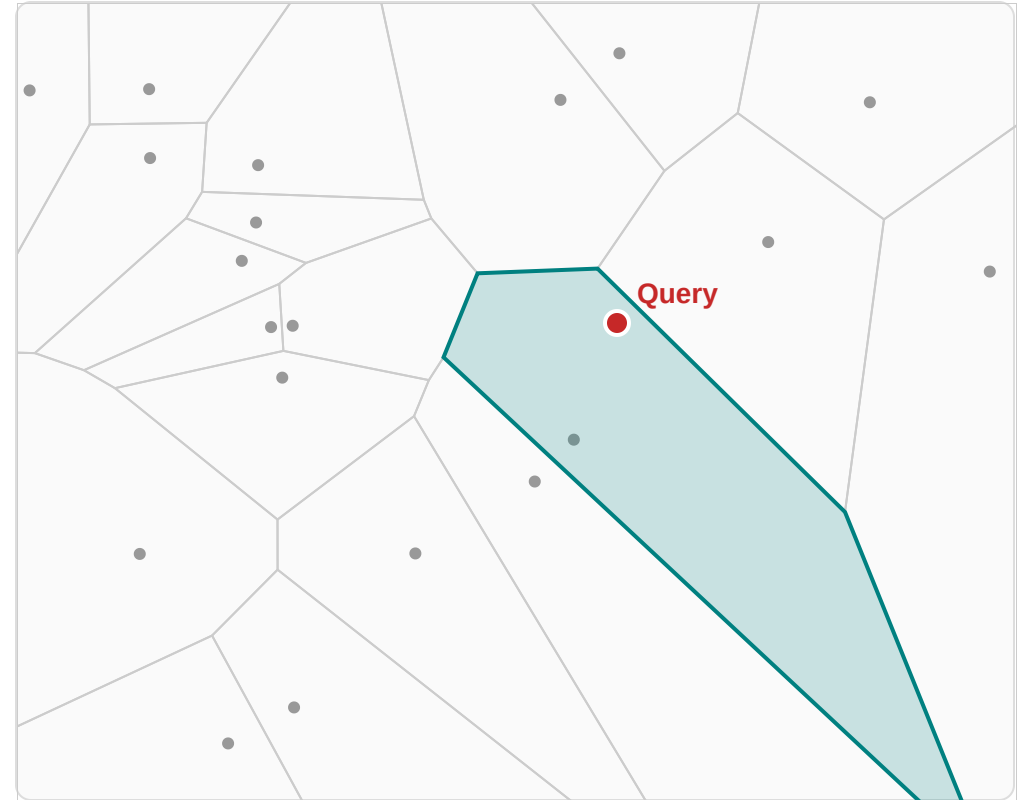
The Trade-off

Accept **95-99% Recall** (miss a few matches)
Gain **100x Speedup** ($O(\log N)$).

FAISS

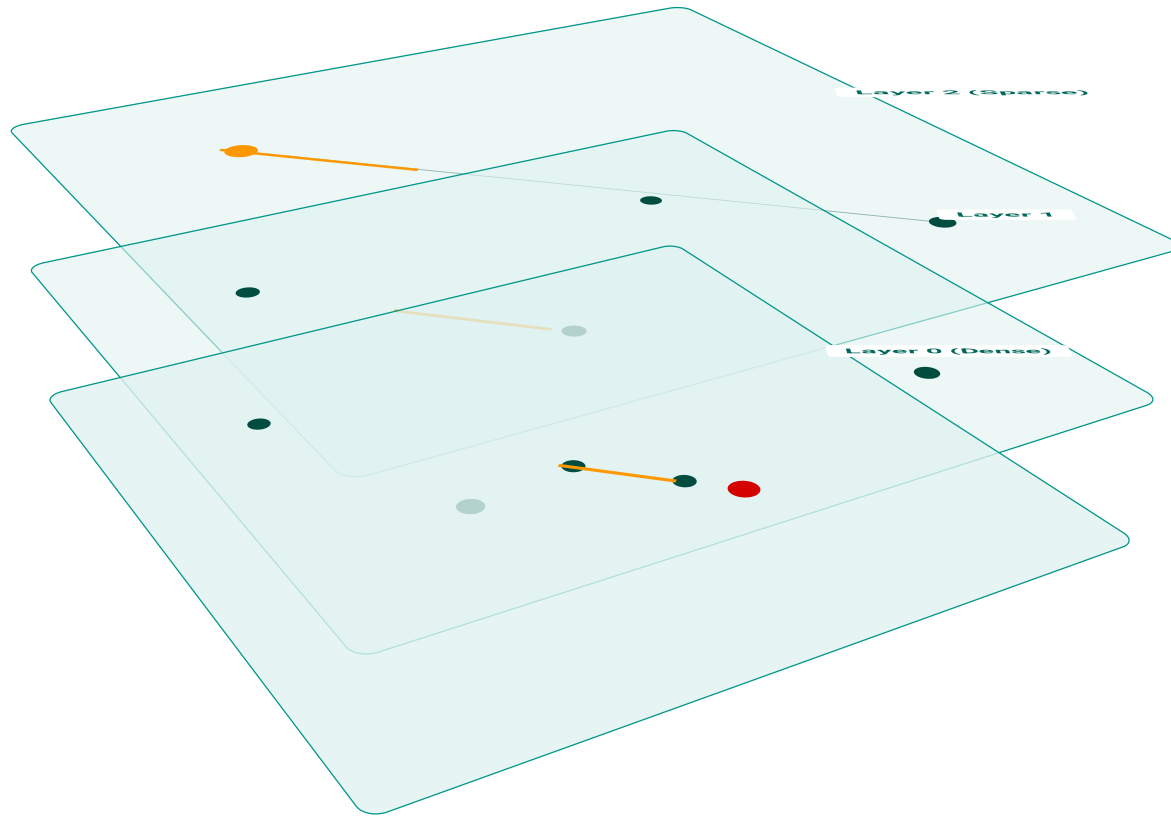
HNSW

ScaNN



Space Partitioning (IVF - Inverted File Index)

HNSW Algorithm



Analogy: "Skip List for Graphs"

1. Zoom In

Start at the top layer (sparse). Like an "Expressway", take long jumps to get roughly to the right neighborhood.

2. Greedy Search

Move to the neighbor closest to the query vector. If no neighbor is closer, drop down to the next layer.

3. Local Refinement

At Layer 0 (dense), perform a fine-grained search to find the exact nearest neighbors.

Complexity: $O(\log N)$

Hybrid Search

🔍 "iPhone 15 pro max case"



Sparse (BM25)

Matches exact keywords.
"iPhone", "15", "Pro"

✓ Best for: Exact Names



Dense (Vector)

Matches semantic meaning.
"phone cover", "protection"

✓ Best for: Concepts



Result Fusion

$$\text{Score} = \alpha \cdot \text{Dense} + (1 - \alpha) \cdot \text{Sparse}$$

Combines precision of keywords with recall of vectors.

Reciprocal Rank Fusion (RRF)

$$\text{RRF}(d) = \sum \frac{1}{k + \text{rank}}$$

BM25 (Sparse)	
1	Doc A
2	Doc B
3	Doc C

Score range: 0 - 25

+

Dense (Vector)	
1	Doc C
2	Doc A
3	Doc D

Score range: 0.0 - 1.0



Fused Result		
1	Doc A	$1/1 + 1/2 = 1.5$
2	Doc C	$1/3 + 1/1 = 1.33$
3	Doc B	$1/2 + 0 = 0.5$

Why RRF?

It ignores the absolute scores (which are incomparable) and relies purely on rank position. No calibration needed!

Retrieval-Augmented Generation (RAG)

Why LLMs Need Help

❄️ Frozen Knowledge

"Who won the Super Bowl yesterday?"
LLM: "I only know up to 2023."

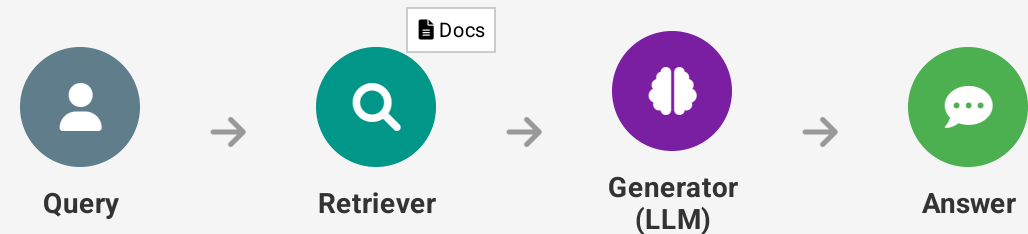
✍️ Hallucination

LLMs are confident liars. They will invent facts if they don't know them.

🏢 Private Data Blindness

ChatGPT doesn't know your company's internal wiki or emails.

*"It's the difference between taking a test from **memory** vs. with an **open textbook**."*



RAG = Search Engine + LLM

RAG Architecture

1. Indexing (Offline)



2. Retrieval & Generation (Online)



Prompt Template

Context: {retrieved_chunks}

Question: {user_query}

Answer the question using ONLY the context above.

The Hallucination Problem

User: "Who won the 2024 Mars Olympics?"

"The 2024 Mars Olympics were won by the United States, with a record 15 gold medals in zero-g swimming."

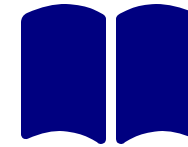
HALLUCINATION

Why? LLMs are "Stochastic Parrots". They predict the next plausible word, not the truth.

Without external knowledge, they fill gaps with convincing fiction.



Pure LLM
(Memory Only)



RAG
(Memory + Search)

Grounding

RAG forces the model to cite its sources.

"According to Document A..."

If the info isn't retrieved, the model can say "I don't know."

The Context Window Bottleneck

📄 Finite Input

You cannot paste your entire company wiki (10GB) into ChatGPT. You must select the most relevant parts.

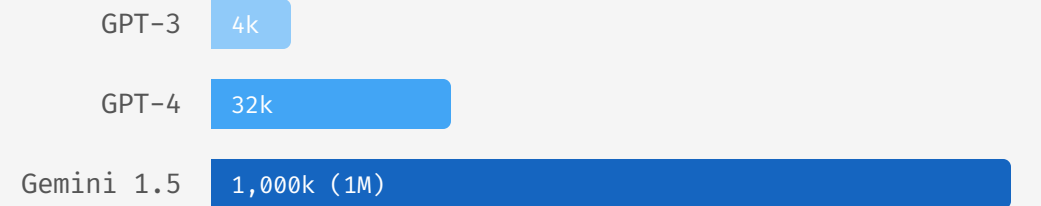
\$ Cost & Latency

Attention is quadratic ($O(N^2)$). Doubling the input length quadruples the cost and processing time.

🎯 "Lost in the Middle"

Research shows LLMs are good at using info at the **start** and **end** of the prompt, but often ignore the middle.

Context Window Evolution



*Log Scale Representation

Even with 1M tokens...

RAG is still needed for *selection, speed, and cost reduction*.

Chunking Strategies

i Why Chunk? LLMs have token limits, and retrieving precise segments yields better answers than retrieving entire documents.

Fixed Size

Split every N tokens (e.g., 512).

Pros: Simple, fast.

Cons: Can cut sentences in half.

```
The quick brown fox jum|ps over the lazy dog.
```

Recursive

Try splitting by paragraph, then sentence, then word.

Pros: Preserves structure.

Cons: Slightly slower.

```
[Para 1] ... | [Para 2] ...
```

Semantic

Split when the *embedding* changes significantly (topic shift).

Pros: Most coherent.

Cons: Expensive to compute.

```
... end of topic A. | Start of topic B...
```

The Importance of Overlap

Always include a "sliding window" (e.g., 10-20%) to ensure context isn't lost at the boundaries.

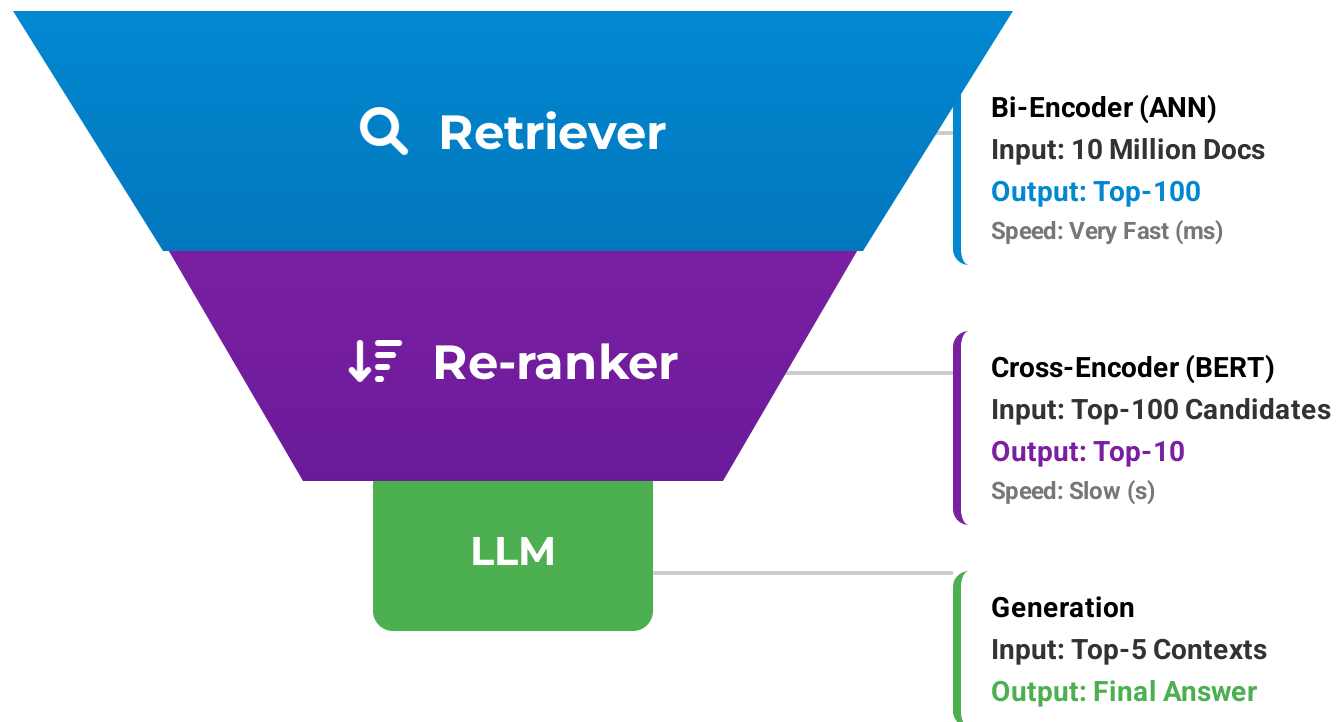
Chunk 1

Shared

Chunk 2

The Re-ranking Step

Two-Stage Retrieval Pipeline



ColBERT: Late Interaction

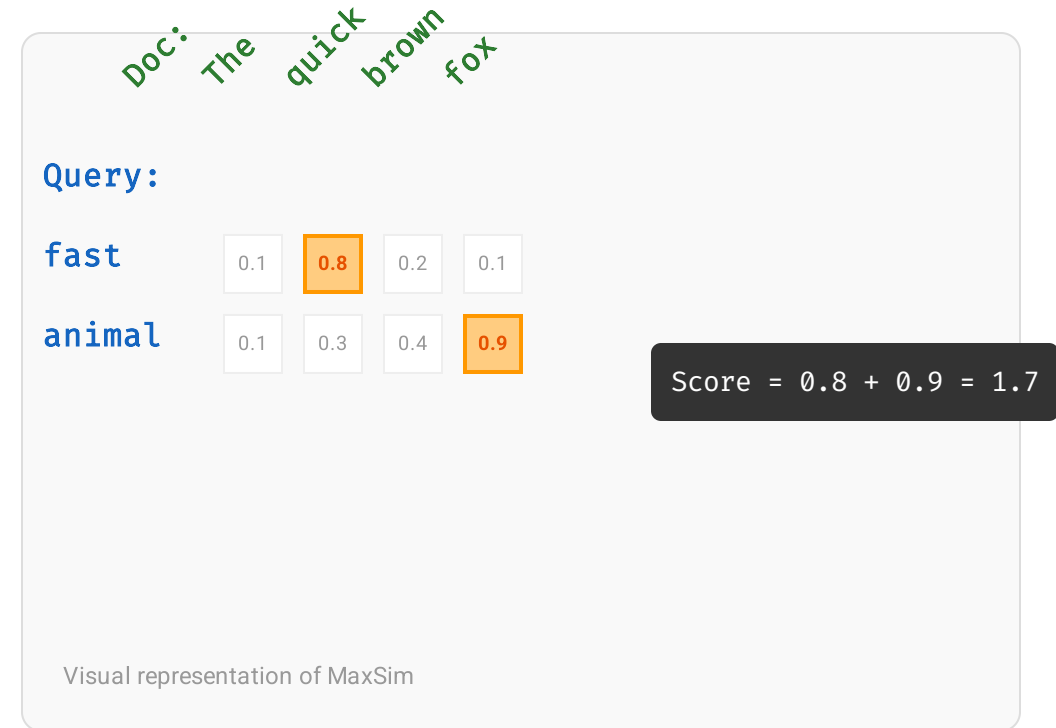
The Innovation

Bi-Encoders squash the whole document into **one vector** (lossy).

ColBERT keeps **all token vectors**. It compares every query token to every document token.

MaxSim Operation:

For each query word, find the *best matching word* in the document, then sum the scores.



"Late Interaction" = Defer the comparison until the end.

SPLADE: Sparse Neural Retrieval

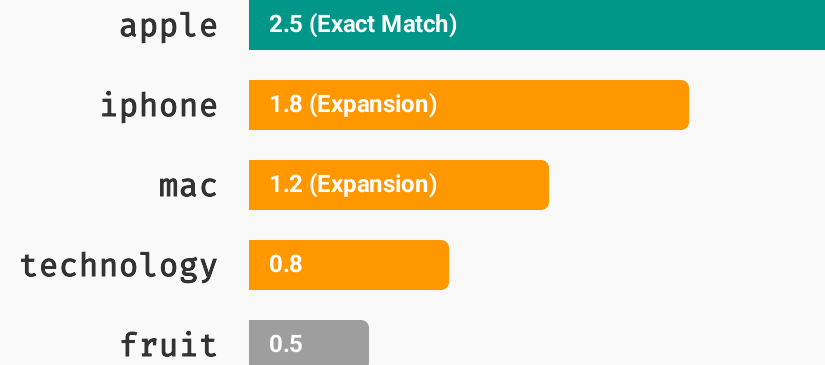
The Idea

Dense vectors (BERT) are opaque floats.
BM25 is clear but lacks semantics.

SPLADE uses BERT to predict *weights* for vocabulary terms, creating a "learned sparse vector".

It performs **Implicit Expansion**: Adding relevant terms that aren't in the query.

Query: "Apple"



Log-Saturation of BERT MLM weights

✓ Compatible with Lucene / Elasticsearch!

Infrastructure: Vector Databases



ANN Indexing

Built-in HNSW / IVF-PQ for fast retrieval.



Metadata Filtering

"Find vectors where year > 2020".



Scaling

Sharding across multiple nodes.

The Landscape



Pinecone

Managed



Weaviate

Open Source



Milvus

Scale-out



Qdrant

Rust-based



pgvector

Postgres

Elasticsearch

kNN Plugin



MongoDB Atlas

Vector Search



FAISS

Library (Meta)

💡 Start Simple: If you have < 1M vectors, pgvector is usually enough.

MTEB Leaderboard

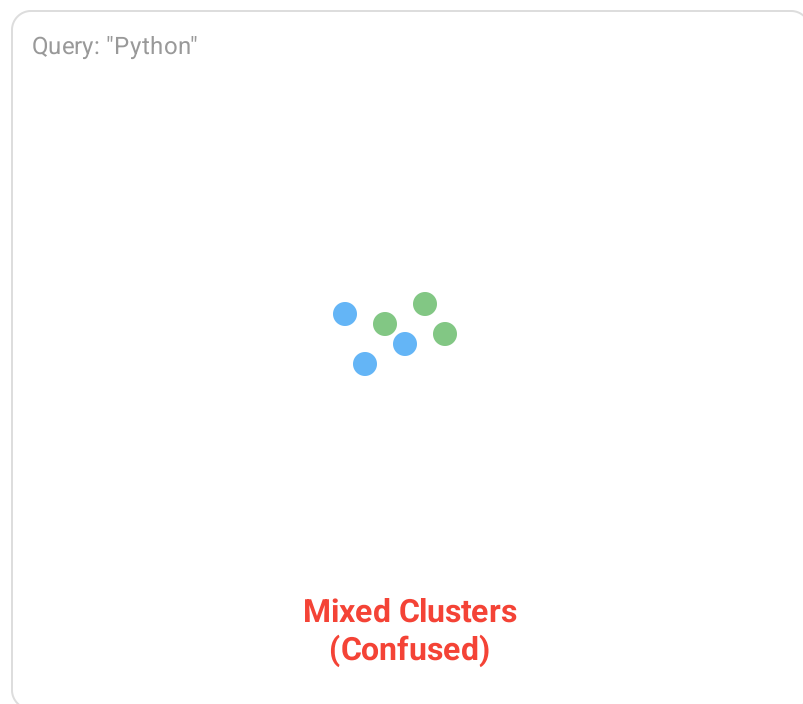
*Massive Text Embedding Benchmark (Average Retrieval Score)

Rank	Model Name	Type	Size	Score (NDCG)
1	OpenAI text-embedding-3-large	API	Unknown	64.6
2	E5-mistral-7b	OPEN	7B Params	63.8
3	BGE-m3	OPEN	560M Params	62.5
15	all-MiniLM-L6-v2	OPEN	22M Params	56.2

 Trade-off: MiniLM is 50x faster and 100x smaller than Mistral, but only ~10% worse. For many real-time apps, MiniLM is still the best choice.

Domain Adaptation: Fine-Tuning

Pre-trained (General)



The model can't distinguish between the programming language and the snake.



Fine-tuned (Domain)

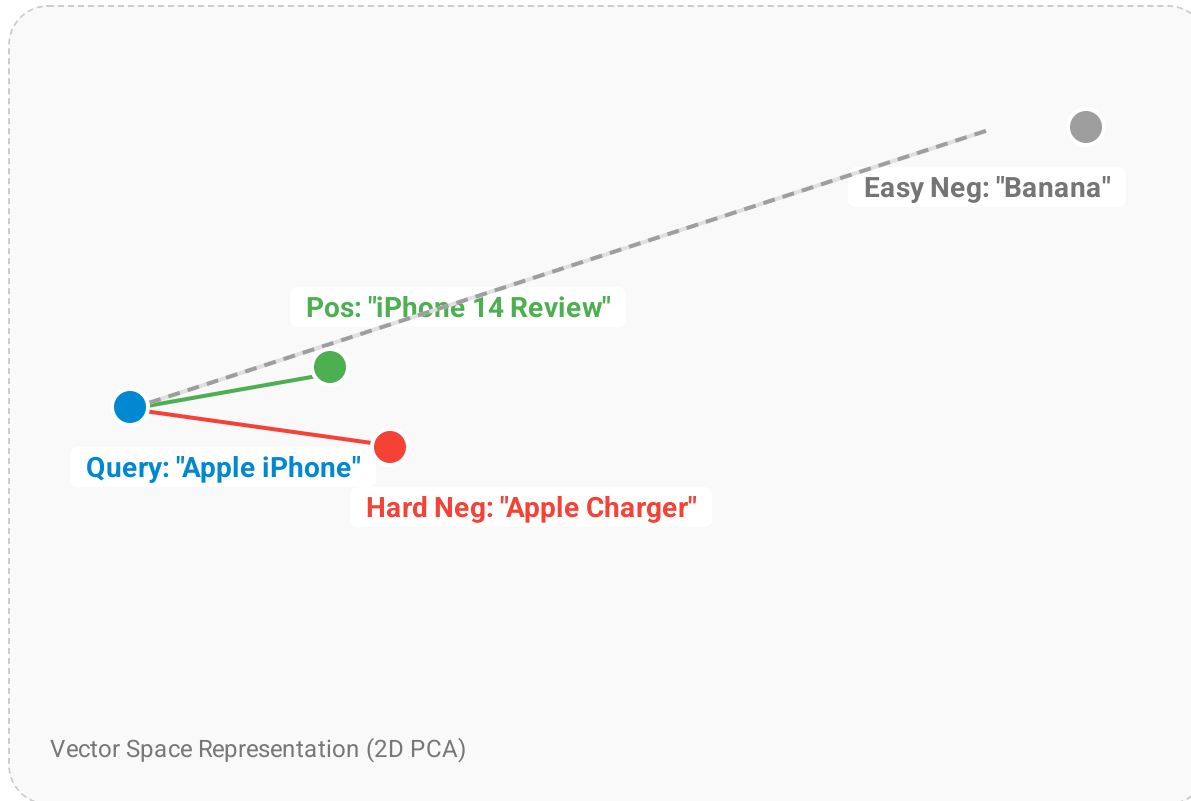


After training on StackOverflow data, the model learns "Python" usually means code in this context.

 **No Training Data? Use GPL!**

Generative Pseudo-Labeling: Use an LLM to generate synthetic queries for your documents, then train on that.

Training: The Importance of Hard Negatives



Easy Negatives

"Banana" is obviously not "iPhone". The model learns **nothing** from this because the distance is already huge.

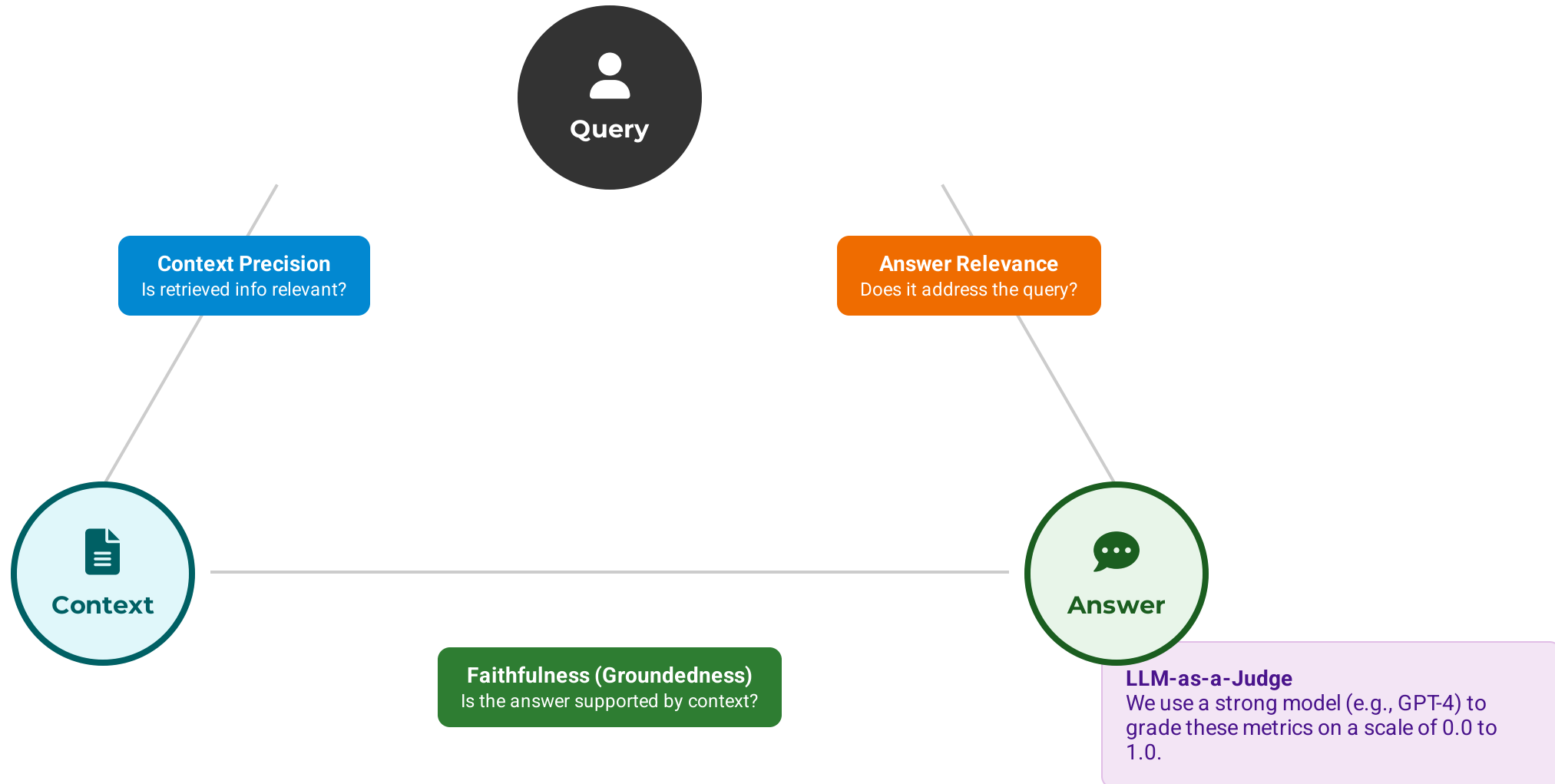
Hard Negatives

"Apple Charger" shares keywords ("Apple") but is semantically wrong. The model **must struggle** to push this away.

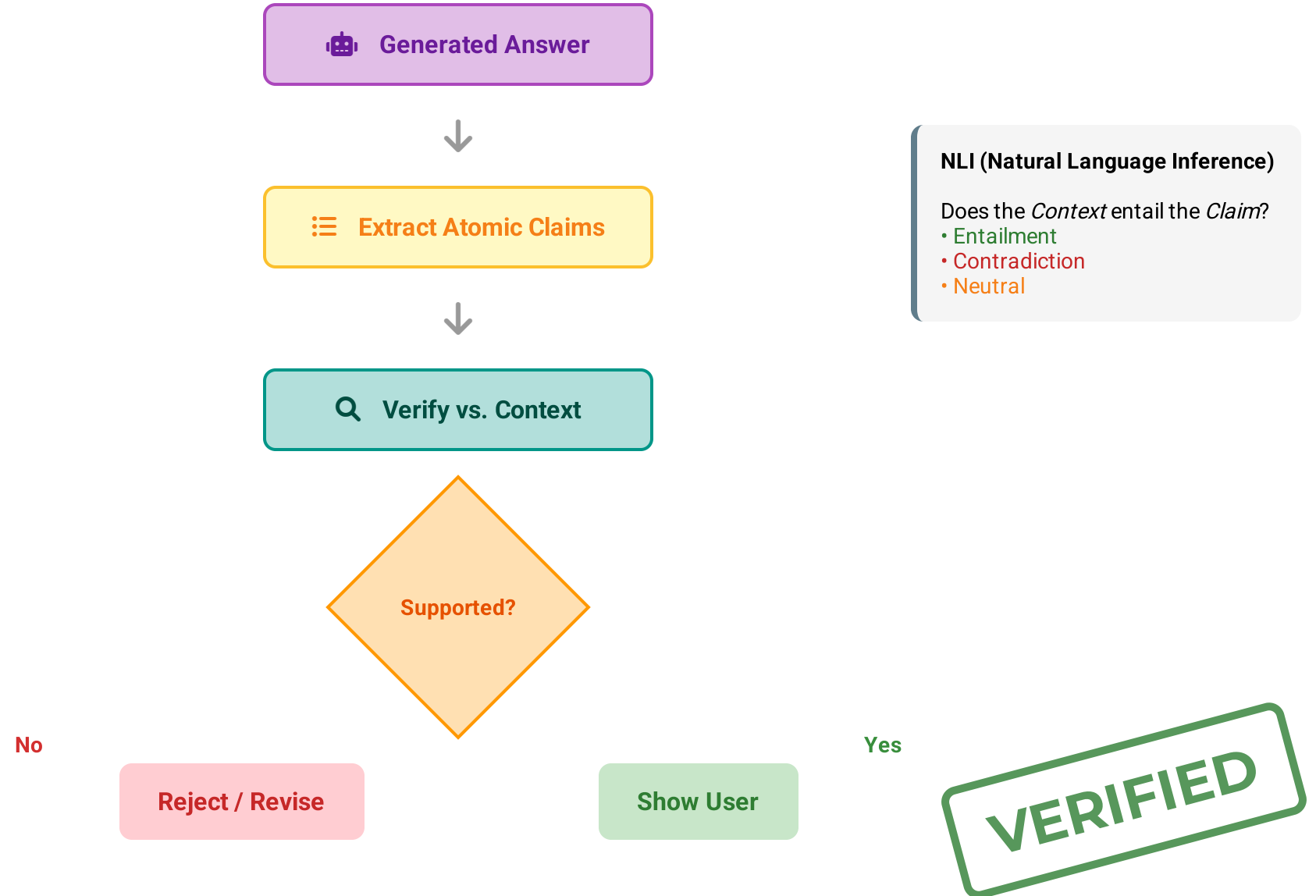
Triplet Loss:

$$L = \max(0, d(Q, P) - d(Q, N) + \text{margin})$$

Evaluating RAG Systems (RAGAS)



Groundedness Checking



Complex Queries: Multi-Hop QA

🔍 "Who is the wife of the actor who played Iron Man?"

HOP 1: SUB-QUERY

"Who played Iron Man?"

📄 Doc A: "Robert Downey Jr. is an actor known for playing Iron Man..."



HOP 2: SUB-QUERY

"Who is Robert Downey Jr.'s wife?"

📄 Doc B: "Susan Downey has been married to Robert since 2005."



FINAL ANSWER

Susan Downey

📄 Standard Benchmark: **HotpotQA** (requires reasoning across multiple Wikipedia paragraphs).

Beyond RAG: Agentic Search

ReAct Paradigm

Reasoning + Acting

RAG is static (Retrieve -> Generate).

Agents are dynamic. They can use tools, run code, and search iteratively until they solve the problem.

Q: "Apple stock * 2?"

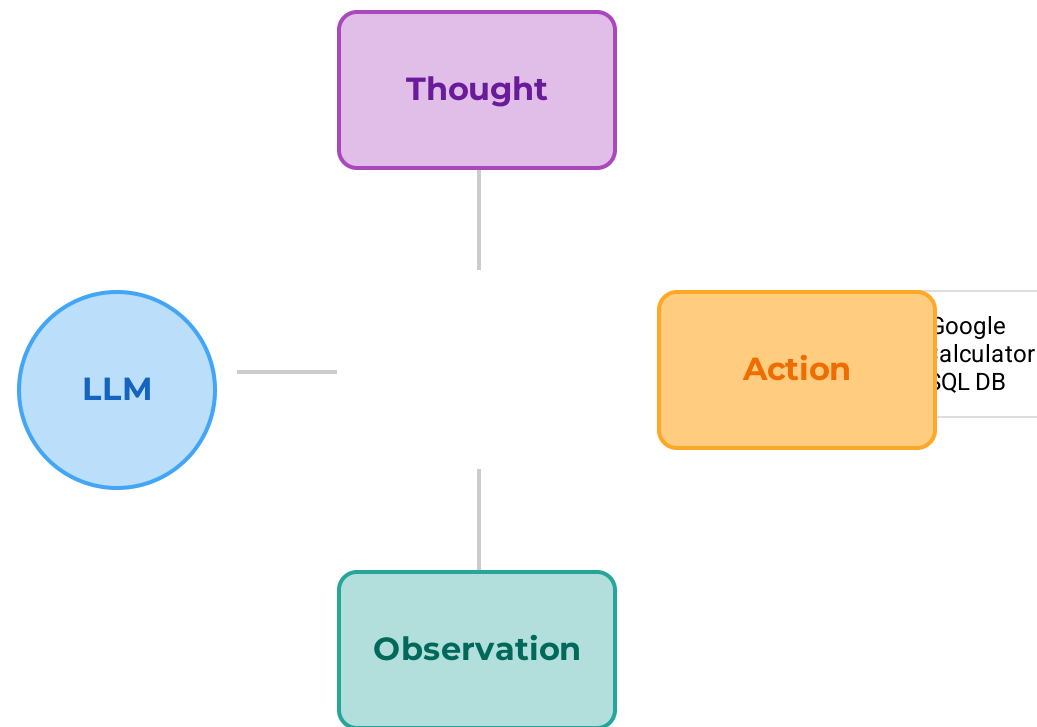
Thought: I need the price.

Action: Search("AAPL") → \$180

Thought: Now multiply.

Action: Calc(180 * 2) → 360

Final Answer: \$360.



"The Agent Loop"

Advanced RAG: Query Expansion

1. HyDE (Hypothetical Doc)

Generate a fake answer, then search with *that*.

User Query:

"golang web server"



LLM Hallucination:

"To start a web server in Go, import net/http and use http.ListenAndServe..."



Search Vector: Use the fake answer vector. It's closer to the real docs!

2. Multi-Query Expansion

Rewrite the query in different ways to cast a wider net.

User Query:

"How to fix 404?"



"HTTP 404 Not Found solutions"

"Server configuration missing file"

"Nginx Apache 404 error logs"



Union: Retrieve docs for ALL 3 queries and de-duplicate.

HyDE (Hypothetical Document Embeddings)



💡 **Key Insight:** Hallucinations can be useful! We don't need the fake answer to be *factually* correct, just *stylistically* similar to the target document.

Self-RAG: Learning to Retrieve & Reflect

🔍 [Retrieve]

✅ [IsRel]

” [IsSup]

★ [IsUse]

Generation Stream

User: Who won the 2022 World Cup?

Model: I need to check. [Retrieve=Yes]

(Retrieves Document: "Argentina defeated France ...")

[IsRel=Relevant] Argentina won the 2022 World Cup. [IsSup=FullySupported]

Key Idea: Train the LLM to output *reflection tokens* alongside text. It learns to critique its own retrieval and generation quality in real-time.

Case Study: Spotify's Semantic Search

The Challenge

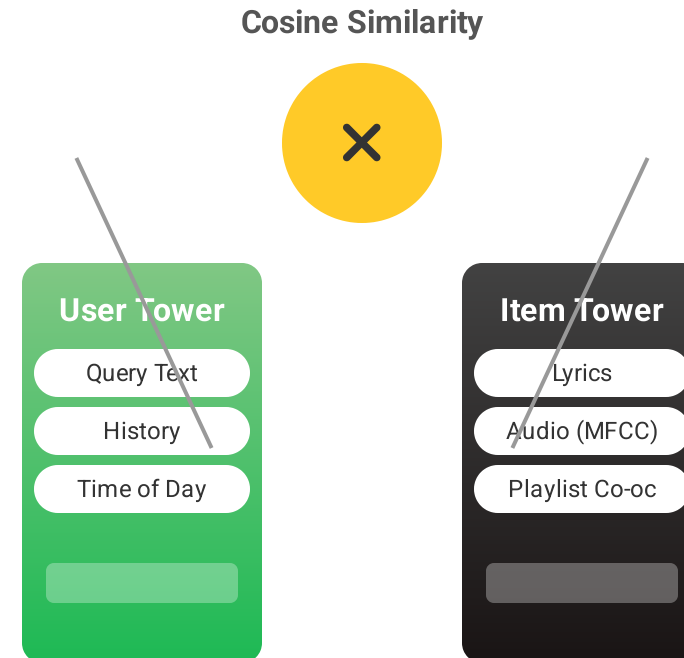
Users search for vibes: *"sad songs for a rainy day"*.

Keyword search fails because the song title might not contain "sad" or "rain".

Solution: Two-Tower Model (DSSM)

Map Users and Songs into the same vector space.

- ✓ Better discovery of long-tail content.
- ✓ Captures audio features + lyrics.



Case Study: Facebook's Semantic Search (EBR)

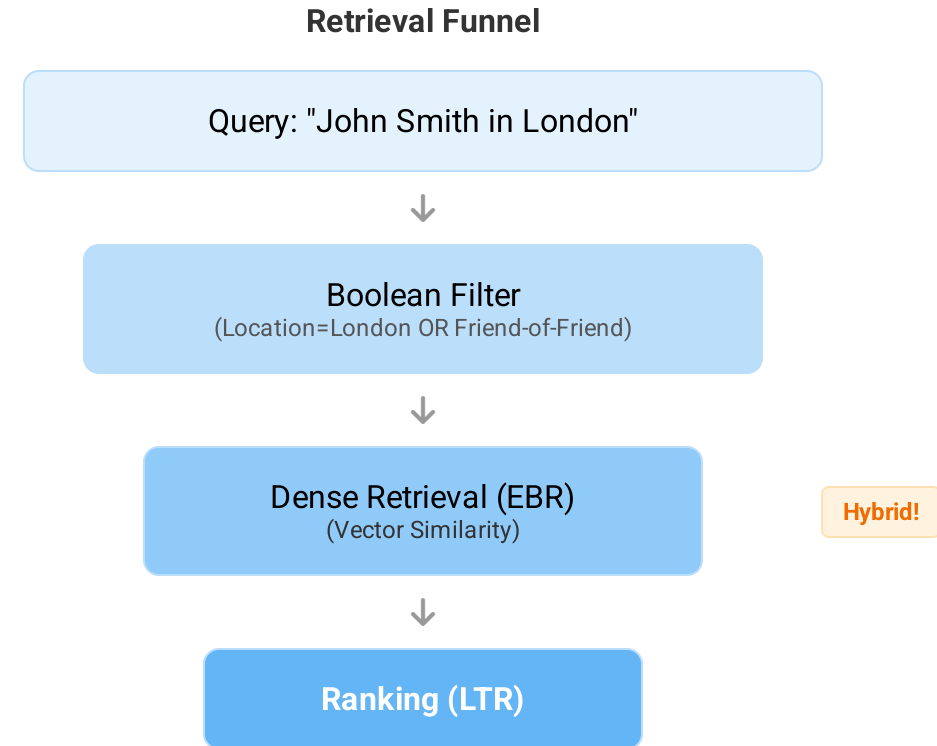
The Challenge

Billions of users. Trillions of posts.
Names are tricky ("Bill" vs "William").
Exact match fails on fuzzy intent.

Solution: Hybrid Search

Combine Boolean filters (Location, Friends) with Dense Retrieval.

💡 **Hard Negative Mining** was the key to success (training on difficult non-matches).



Source: Huang et al., "Embedding-based Retrieval in Facebook Search", KDD 2020.

Challenges in Neural IR



Domain Shift (Generalization)

Models trained on MS MARCO (web queries) often fail on specialized domains like Medical (BioASQ) or Legal (COLIEE) due to vocabulary mismatch.

🔧 Fix: Domain Adaptation / Fine-tuning (GPL)



Latency & Cost

Cross-Encoders are highly accurate but prohibitively slow for large-scale retrieval. Running BERT on millions of docs is not feasible in real-time.

🔧 Fix: Distillation, Late Interaction (ColBERT)



Interpretability (Black Box)

Why did the model retrieve this document? Unlike BM25 (where you can see term overlap), vector similarity is opaque and hard to debug.

🔧 Fix: Sparse Neural Models (SPLADE)

Interpretability: Why did it retrieve that?

1. Attention Maps (BERT)

Visualizing which words the model focused on.

The quick brown fox jumps
over the lazy dog.

❗ Hard for humans to interpret fully.

2. Sparse Expansion (SPLADE)

Projecting dense vectors back to readable words.

Query: "Apple"

iPhone	
Mac	
Tech	

✅ Human-readable!

3. Counterfactuals

"What if?" analysis.

Original: "Bank of America"
(Rank 1)

Perturbed: "River Bank of America"
(Rank 50)

Shows the model is sensitive to context "River".

"Trust requires Explainability."

The Future of Neural IR



Multimodal Retrieval

Search isn't just text anymore. Models like **CLIP** map images, audio, and video into the same vector space, enabling cross-modal search ("Find the video where the dog jumps").



Long-Context LLMs

With 1M+ token windows (Gemini 1.5), do we need RAG? Yes, for cost and latency, but the boundary is shifting. "Retrieval" becomes "In-Context Learning".



Personalized Search

Adapting dense vectors to user history. Your "Apple" (Tech) is different from a chef's "Apple" (Fruit).

"Search is evolving from **Keyword Matching** to **Intent Understanding**."

Lab: Using Hugging Face Transformers

Transformers

```
$ pip install transformers
```

- ✓ The industry standard for NLP.
- ✓ Access to 500k+ models (BERT, GPT, Llama).
- ✓ Unified API for PyTorch & TensorFlow.

```
from transformers import pipeline

# 1. Load a pre-trained pipeline
classifier = pipeline("sentiment-analysis")

# 2. Run inference
result = classifier("I love this IR course!")
print(result)
```

```
[{'label': 'POSITIVE', 'score': 0.9998}]
```

We will use the sentence-transformers wrapper for easier embedding generation.

Code: Semantic Search with FAISS

python

```
1 from sentence_transformers import SentenceTransformer
2 import faiss
3
4 # 1. Load Model & Encode Docs
5 model = SentenceTransformer('all-MiniLM-L6-v2')
6 docs = ["The cat sits", "A man walks", "Machine learning"]
7 embeddings = model.encode(docs)
8
9 # 2. Build FAISS Index
10 d = embeddings.shape[1] # Dimension (384)
11 index = faiss.IndexFlatL2(d)
12 index.add(embeddings)
13
14 # 3. Search
15 query = model.encode(["artificial intelligence"])
16 D, I = index.search(query, k=1)
17 print(docs[I[0][0]]) # Output: "Machine learning"
```

Step 1

Encode

We use a pre-trained model (MiniLM) to convert text into dense vectors (384 dimensions).

Step 2

Index

IndexFlatL2 performs exact Euclidean distance search. For large datasets, use IndexIVFFlat.

Step 3

Search

Find the nearest neighbor. "AI" matches "Machine learning" despite no keyword overlap!

Code: Simple RAG Pipeline

python

```
1 def rag_pipeline(query, index, model, llm):
2     # 1. Retrieve Context
3     q_emb = model.encode([query])
4     _, I = index.search(q_emb, k=3)
5     context = [docs[i] for i in I[0]]
6
7     # 2. Augment Prompt
8     prompt = f"""
9     Context: {context}
10    Question: {query}
11    Answer based ONLY on the context:
12    """
13
14    # 3. Generate Answer
15    response = llm.generate(prompt)
16    return response
```

Step 1

Retrieve

We reuse the FAISS index from the previous slide to find the top-3 most relevant chunks.

Step 2

Augment

We inject the retrieved text into the prompt. The instruction "Answer based ONLY on context" is crucial for grounding.

Step 3

Generate

The LLM acts as a synthesizer, reading the messy chunks and writing a fluent answer.

Summary: The Neural Shift

Sparse (Symbolic)

- ✓ **BM25 / TF-IDF:** Fast, explainable, exact matches.
- ✗ **Flaw:** Fails at synonyms ("car" ≠ "auto") and polysemy.

Dense (Neural)

- ✓ **Embeddings (BERT):** Captures semantic meaning and context.
- ✓ **Bi-Encoders:** Enable fast retrieval via ANN (FAISS).

The Modern Stack

-  **Hybrid Search:**
Combine BM25 + Vectors using Reciprocal Rank Fusion (RRF).
-  **Re-ranking:**
Use Cross-Encoders to refine the top-k results.
-  **RAG:**
Ground LLMs in retrieved facts to solve hallucinations.

Next Week: Learning to Rank (LTR) →

Required Reading



Introduction to Information Retrieval

Manning, Raghavan, Schütze (2008) - Chapter 6 & 7

Textbook



BERT: Pre-training of Deep Bidirectional Transformers

Devlin et al. (2018)

Seminal Paper



Retrieval-Augmented Generation for Knowledge-Intensive NLP

Lewis et al. (2020)

RAG Origin



ColBERT: Efficient and Effective Passage Search

Khattab & Zaharia (2020)

Advanced IR

💡 Focus on the "Model Architecture" and "Experiments" sections. Skip the proofs.

Discussion Questions



When should you use Sparse (BM25) vs. Dense (Vectors)?

Think about: Exact keywords (IDs, codes) vs. Conceptual queries.



Why are Hard Negatives crucial for training Bi-Encoders?

What does the model learn from "easy" negatives? (Gradient magnitude).



How does RAG solve the hallucination problem?

Think about "Grounding" and "Source Citation".



What is the main trade-off of ColBERT vs. Bi-Encoders?

Think about Storage (Index size) and Latency vs. Accuracy.